

Subprogramas em VHDL

Funções e procedimentos aplicados a timers, displays e VGA

Aula prática de VHDL

Base: arquivos Aula009

Roteiro da aula

1. Timer sem procedimento: onde aparece a repetição.
2. Testbench do timer: clock, reset e simulação rápida.
3. Timer com procedimento: reutilização da lógica de incremento.
4. Função para display de sete segmentos.
5. Integração no top-level para a DE10-Lite.
6. Exemplo VGA: procedimentos para varredura e desenho.
7. Labs e boas práticas.

O que são subprogramas em VHDL?

Subprogramas são blocos de código nomeados que podem ser reutilizados dentro de uma arquitetura, pacote, processo ou outro escopo declarativo.

Em VHDL, os dois tipos principais são:

- **Função:** calcula e retorna um único valor.
- **Procedimento:** executa uma sequência de comandos e pode alterar sinais, variáveis ou produzir múltiplos resultados.

Nesta aula, a ideia não é usar subprogramas por estilo, mas porque os exemplos mostram uma necessidade real: evitar repetição e deixar explícita a intenção do hardware descrito.

Função ou procedimento?

Critério	Função	Procedimento
Resultado	Retorna um valor	Pode ter várias saídas
Uso típico	Expressões e conversões	Sequências reutilizáveis de comandos
Efeito esperado	Sem efeito colateral	Pode alterar sinais ou variáveis
Exemplo da aula	Decodificar um dígito	Incrementar contador com estouro

Regra prática: se a operação parece uma expressão, use função. Se parece uma pequena rotina de controle, use procedimento.

Classes de objetos em parâmetros

Em subprogramas, o parâmetro formal pode indicar a classe do objeto recebido. Funções normalmente usam entradas; procedimentos podem também alterar sinais ou variáveis.

Função

```
1 function decodificando(  
2     valor : integer  
3 ) return std_logic_vector is  
4 begin  
5     -- calcula e retorna um valor  
6 end function;
```

Procedimento

```
1 procedure incrementa(  
2     signal contador : inout unsigned;  
3     constant max      : in      positive;  
4     constant habilita : in      boolean;  
5     variable estouro  : out     boolean  
6 ) is
```

- Em função, valor : integer significa constant valor : in integer; normalmente omitimos o padrão.
- variable não pode ser usada como classe de parâmetro formal em funções.
- signal: conectado a um sinal do circuito; atribuições usam <=.
- Em procedimentos, variable é útil para resultado imediato dentro de um processo; atribuições usam :=.
- file: existe em VHDL, mas é voltado a simulação e não é usado nestes exemplos.

Funções e procedimentos: modos

Além da classe do objeto, o parâmetro tem um modo:

- **in**: o subprograma lê o valor.
- **out**: o subprograma produz um valor.
- **inout**: o subprograma lê o valor atual e também escreve um novo valor.

Em uma **função**, os parâmetros são entradas e a saída principal é o **return**. Por isso ela combina bem com expressões, como o decodificador de sete segmentos.

Em um **procedimento**, **out** e **inout** são naturais: é o caso de **incrementa**, que altera o contador e devolve a flag de estouro.

Chamar dentro ou fora de um process

Dentro de um process, a chamada participa da sequência daquele processo. No timer, isso é essencial porque as chamadas acontecem na borda de clock.

```
1 elsif falling_edge(clk) then
2     incrementa(ticks, clockFreq, true, controle);
3     incrementa(segReg, 60, controle, controle);
4     incrementa(minReg, 60, controle, controle);
5     incrementa(horReg, 24, controle, controle);
6 end if;
```

Fora de um process, funções aparecem naturalmente em atribuições concorrentes:

```
1 d0 <= decodificando(unidade);
```

Procedimentos também podem ter chamada concorrente em VHDL, mas em projetos didáticos e sintetizáveis costuma ser mais claro chamá-los dentro de processos.

Exemplo inicial: timer sem procedimento

O arquivo `a009a_timer.vhd` descreve um timer com um contador interno de ticks e três contadores visíveis na interface.

```
1 port(  
2     clk  : in  std_logic;  
3     nRst : in  std_logic;  
4     s    : out unsigned(5 downto 0);  
5     m    : out unsigned(5 downto 0);  
6     h    : out unsigned(4 downto 0)  
7 );
```

```
1 architecture rtl of a009a_timer is  
2     signal ticks  : unsigned(bits_necessarios(clockFreq) - 1 downto 0) := (others => '0');  
3     signal segReg : unsigned(5 downto 0) := (others => '0');  
4     -- minReg e horReg seguem a mesma ideia  
5 begin  
6     -- logica sequencial do timer
```

`ticks` pertence à arquitetura: divide o clock até formar 1 segundo. Já `s`, `m` e `h` são portas out, conectáveis ao testbench, ao top-level ou aos displays.

Reset e contador de ticks

O processo primeiro trata o reset assíncrono ativo em nível baixo. Depois, a cada borda de descida, ticks transforma ciclos de clock em segundos.

```
1  if nRst = '0' then
2      ticks <= (others => '0');
3      segReg <= (others => '0');
4      minReg <= (others => '0');
5      horReg <= (others => '0');
6
7  elsif falling_edge(clk) then
8      if ticks = to_unsigned(clockFreq - 1, ticks'length) then
9          ticks <= (others => '0');
10         -- aqui entram segundos, minutos e horas
11     else
12         ticks <= ticks + 1;
13     end if;
14 end if;
```

Em simulação, clockFreq pode ser pequeno, como 10 Hz. No hardware real da DE10-Lite, o clock principal é de 50 MHz.

Incremento em cascata

Quando ticks estoura, o timer incrementa segundos. Se segundos estouram, incrementa minutos. Se minutos estouram, incrementa horas.

```
1  if segReg = to_unsigned(59, segReg'length) then
2      segReg <= (others => '0');
3
4      if minReg = to_unsigned(59, minReg'length) then
5          minReg <= (others => '0');
6
7          if horReg = to_unsigned(23, horReg'length) then
8              horReg <= (others => '0');
9          else
10             horReg <= horReg + 1;
11         end if;
12     else
13         minReg <= minReg + 1;
14     end if;
15 else
16     segReg <= segReg + 1;
17 end if;
```

A estrutura funciona, mas o padrão “incrementa ou volta para zero” aparece várias vezes.

O que este código nos ensina?

O timer sem procedimento é um bom primeiro passo porque deixa a lógica visível:

- existe um contador base;
- cada contador possui um limite;
- ao chegar ao limite, ele volta para zero;
- o estouro de um contador habilita o próximo.

A motivação para o procedimento nasce daqui: a lógica de incremento com estouro é a mesma para `ticks`, `s`, `m` e `h`; mudam apenas o limite e a condição de habilitação.

Testbench do timer

O arquivo `a009b_timer_tb.vhd` instancia o timer original e reduz a frequência para tornar a simulação curta.

```
1 constant clockFreq : positive := 10;
2 constant clkPer    : time := 1000 ms / clockFreq;
3
4 signal clk, nRst : std_logic := '0';
5 signal segundo   : unsigned(5 downto 0);
6 signal minuto    : unsigned(5 downto 0);
7 signal hora      : unsigned(4 downto 0);
```

Em vez de esperar 50 milhões de ciclos para ver um segundo passar, o testbench usa 10 ciclos por segundo simulado e verifica automaticamente o primeiro transbordo de segundos para minutos.

Instanciação do DUT

O DUT é o *device under test*: o circuito que queremos exercitar na simulação.

```
1 i_timer: entity work.a009a_timer(rtl)
2   generic map (
3       clockFreq => clockFreq
4   )
5   port map (
6       clk  => clk,
7       nRst => nRst,
8       s    => segundo,
9       m    => minuto,
10      h    => hora
11  );
```

O `generic map` troca o valor padrão do projeto por um valor adequado à simulação.

Clock e reset no testbench

O clock é gerado por uma atribuição concorrente. O reset começa ativo e depois é liberado.

```
1  clk <= not clk after clkPer / 2;
2
3  process is
4  begin
5      wait until rising_edge(clk);
6      wait until rising_edge(clk);
7
8      nRst <= '1';
9
10     wait;
11 end process;
```

Mesmo que o DUT use `falling_edge(clk)`, o testbench aguarda duas bordas de subida apenas para dar um intervalo inicial antes de liberar o reset.

Timer com procedimento

O arquivo a009c_timer_proc.vhd preserva a ideia do timer, mas encapsula o padrão de incremento.

```
1 procedure incrementa(  
2     signal  contador : inout unsigned;  
3     constant max      : in      positive;  
4     constant habilita : in      boolean;  
5     variable estouro  : out     boolean  
6 ) is  
7 begin  
8     estouro := false;  
9     if habilita then  
10         if contador = to_unsigned(max - 1, contador'length) then  
11             contador <= to_unsigned(0, contador'length);  
12             estouro := true;  
13         else  
14             contador <= contador + 1;  
15         end if;  
16     end if;  
17 end procedure;
```

Este procedimento descreve o comportamento: incrementar se estiver habilitado e avisar quando

Por que estes parâmetros?

- `signal contador : inout unsigned`: o procedimento altera um registrador do circuito, como `ticks`, `segReg`, `minReg` ou `horReg`.
- `constant max : in positive`: o limite é apenas lido; não deve ser modificado.
- `constant habilita : in boolean`: a condição de avanço também é apenas lida.
- `variable estouro : out boolean`: a resposta é usada imediatamente dentro do processo para habilitar o próximo contador.

A classe do objeto importa: sinais modelam conexões registradas ou combinacionais; variáveis dentro do processo atualizam imediatamente no fluxo sequencial.

Chamadas em cascata

No processo principal, a mesma variável controle carrega o estouro de um contador para o próximo.

```
1 process(clk, nRst) is
2     variable controle : boolean;
3 begin
4     if nRst = '0' then
5         ticks <= (others => '0');
6         segReg <= (others => '0');
7         minReg <= (others => '0');
8         horReg <= (others => '0');
9
10    elsif falling_edge(clk) then
11        incrementa(ticks, clockFreq, true, controle);
12        incrementa(segReg, 60, controle, controle);
13        incrementa(minReg, 60, controle, controle);
14        incrementa(horReg, 24, controle, controle);
15    end if;
16 end process;
```

A leitura é quase uma frase: incrementa ticks sempre; se estourar, incrementa segundos; se estourar, incrementa minutos; se estourar, incrementa horas.

Comparação: antes e depois

Timer sem procedimento	Timer com procedimento
Mostra toda a lógica explicitamente. Usa vários if aninhados. É fácil de acompanhar no primeiro contato. A repetição aparece no corpo do processo.	Dá nome à lógica repetida. Usa chamadas sequenciais em cascata. É mais fácil de manter e ampliar. A repetição fica concentrada em incrementa.

O procedimento não cria uma “função de software em execução”. Ele é uma forma de escrever a descrição do hardware de maneira organizada.

Observações de síntese e estilo

- O procedimento é sintetizável porque contém lógica compatível com hardware: comparações, atribuições e controle simples.
- Com `unsigned`, a largura do contador fica explícita e as comparações usam `to_unsigned`.
- As portas são out; os valores são guardados em sinais internos como `segReg`, `minReg` e `horReg`.
- Usar subprogramas não remove a necessidade de pensar em registradores, bordas de clock e reset.

Testbench do timer com procedimento

O arquivo a009f_timer_proc_tb.vhd repete a estratégia do testbench anterior.

```
1 constant clockFreq : positive := 10;
2 constant clkPer    : time := 1000 ms / clockFreq;
3
4 signal clk, nRst : std_logic := '0';
5 signal segundo   : unsigned(5 downto 0);
6 signal minuto    : unsigned(5 downto 0);
7 signal hora      : unsigned(4 downto 0);
```

A interface observada é a mesma em intenção: clock, reset, segundos, minutos e horas. Agora os contadores aparecem como vetores unsigned.

Mesmo teste, outra implementação

```
1  i_timer : entity work.a009c_timer_proc(rtl)
2      generic map(
3          clockFreq => clockFreq
4      )
5      port map(
6          clk  => clk,
7          nRst => nRst,
8          s    => segundo,
9          m    => minuto,
10         h    => hora
11     );
```

Isso é uma ideia importante em projeto digital: podemos refatorar a implementação interna mantendo a interface. O testbench ajuda a verificar se o comportamento externo continua coerente.

Função para display de sete segmentos

O arquivo `a009d_decod.vhd` usa uma função para converter um valor decimal em padrão de segmentos.

```
1 function decodificando(valor: integer)
2     return std_logic_vector is
3
4     variable saida : std_logic_vector(0 to 6);
5 begin
6     case valor is
7         when 0 => saida := not "1111110";
8         when 1 => saida := not "0110000";
9         when 2 => saida := not "1101101";
10        -- ...
11        when 9 => saida := not "1111011";
12        when others => saida := "0000000";
13    end case;
14
15    return saida;
16 end decodificando;
```

Aqui a função é natural: entra um inteiro, sai um vetor de sete bits.

Por que função é adequada aqui?

A decodificação do display tem uma característica simples:

- não precisa alterar sinais por conta própria;
- não precisa devolver vários resultados;
- representa uma tabela de conversão;
- pode ser usada diretamente em atribuições.

A função decodificando transforma uma decisão combinacional em um bloco nomeado. Isso melhora a leitura do processo que separa dezena e unidade.

Decomposição em dezena e unidade

O processo do decodificador separa o número decimal em dois dígitos.

```
1 process(contador) is
2     variable valor    : integer range 0 to 63;
3     variable dezena   : integer range 0 to 6;
4     variable unidade  : integer range 0 to 9;
5 begin
6     valor    := to_integer(contador);
7     dezena   := valor / 10;
8     unidade  := valor mod 10;
9
10    d1 <= decodificando(dezena);
11    d0 <= decodificando(unidade);
12 end process;
```

to_integer converte o unsigned para cálculo decimal. Depois, / faz divisão inteira e mod calcula o resto.

Detalhe do hardware da placa

Os padrões são escritos com not "1111110", not "0110000" e assim por diante.

Isso indica uma característica comum dos displays de sete segmentos em kits FPGA: os segmentos podem ser ativos em nível baixo. Nesse caso:

- bit em '0' acende o segmento;
- bit em '1' apaga o segmento;
- o not adapta uma tabela pensada como ativo alto para a saída ativa baixa.

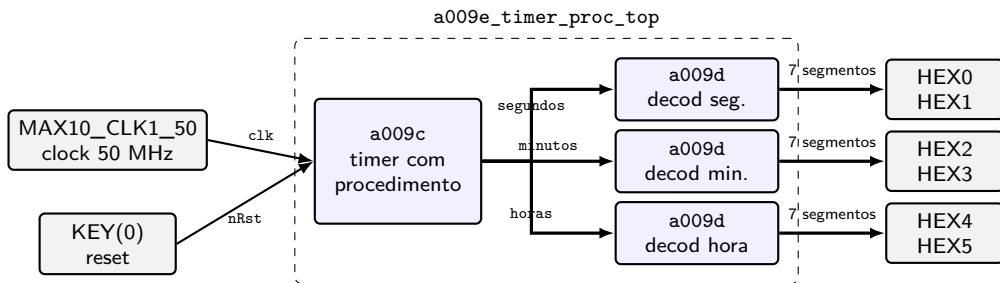
Integração no top-level

O arquivo `a009e_timer_proc_top.vhd` combina o timer com três instâncias do decodificador.

```
1  entity a009e_timer_proc_top is
2      generic(
3          clockFreq : positive := 50e6
4      );
5      port(
6          MAX10_CLK1_50 : in  std_logic;
7          KEY            : in  std_logic_vector(0 downto 0);
8          HEX0, HEX1     : out std_logic_vector(0 to 6);
9          HEX2, HEX3     : out std_logic_vector(0 to 6);
10         HEX4, HEX5     : out std_logic_vector(0 to 6)
11     );
12 end entity;
```

Esta entidade é a ponte entre o projeto VHDL e os pinos do kit DE10-Lite.

Integração do timer no top-level



O top-level não calcula o tempo nem decodifica segmentos: ele organiza as conexões entre os blocos.

Três decodificadores

Cada contador do timer alimenta um par de displays.

```
1  decod_s: entity work.a009d_decod(rtl)
2      port map(contador => segundos, d0 => HEX0, d1 => HEX1);
3
4  decod_m: entity work.a009d_decod(rtl)
5      port map(contador => minutos, d0 => HEX2, d1 => HEX3);
6
7  decod_h: entity work.a009d_decod(rtl)
8      port map(contador => resize(horas, 6), d0 => HEX4, d1 => HEX5);
```

A reutilização aparece em dois níveis: a função é reutilizada dentro do decodificador, e o módulo decodificador é reutilizado três vezes no top-level.

Timer conectado à DE10-Lite

```
1 timer_0: entity work.a009c_timer_proc(rtl)
2     generic map(
3         clockFreq => clockFreq
4     )
5     port map(
6         clk  => MAX10_CLK1_50,
7         nRst => KEY(0),
8         s    => segundos,
9         m    => minutos,
10        h    => horas
11    );
```

O projeto completo usa o clock de 50 MHz da placa, o botão KEY(0) como reset ativo baixo e os displays HEX0 a HEX5 para mostrar segundos, minutos e horas.

Testbench do top-level

O arquivo `a009e_timer_proc_top_tb.vhd` testa a integração completa com frequência reduzida.

```
1 dut: entity work.a009e_timer_proc_top(main)
2   generic map(clockFreq => 10)
3   port map(
4       MAX10_CLK1_50 => clk,
5       KEY            => key,
6       HEX0 => HEX0, HEX1 => HEX1,
7       HEX2 => HEX2, HEX3 => HEX3,
8       HEX4 => HEX4, HEX5 => HEX5
9   );
```

O teste verifica padrões nos displays: primeiro 00:00:01, depois 00:01:00. Assim, ele cobre timer, decodificadores e conexões do top-level.

O que o top-level mostra sobre organização?

O top-level não deveria conter todos os detalhes internos do timer nem todos os detalhes da tabela de sete segmentos.

Seu papel é conectar blocos:

- o timer produz valores numéricos;
- os decodificadores transformam valores em segmentos;
- os pinos da placa recebem clock, reset e sinais de display.

Subprogramas ajudam dentro dos módulos. Instanciação de entidades ajuda entre módulos. As duas ideias se complementam.

Exemplo VGA

O arquivo a009j_vga.vhd gera três quadrados coloridos em um monitor VGA 800x600 a partir do clock de 50 MHz da DE10-Lite.

```
1 entity a009j_vga is
2     port (
3         MAX10_CLK1_50 : in  std_logic;
4         VGA_R          : out std_logic_vector(3 downto 0);
5         VGA_G          : out std_logic_vector(3 downto 0);
6         VGA_B          : out std_logic_vector(3 downto 0);
7         VGA_VS, VGA_HS : out std_logic
8     );
9 end entity;
```

Este exemplo usa procedimentos para separar duas preocupações: varredura da tela e desenho.

Constantes de temporização

As constantes definem os intervalos horizontal e vertical: área visível, *front porch*, pulso de sincronismo, *back porch* e total.

```
1 constant H_ACTIVE_VIDEO : integer := 800;
2 constant H_FRONT_PORCH  : integer := H_ACTIVE_VIDEO + 56;
3 constant H_SYNC          : integer := H_FRONT_PORCH + 120;
4 constant H_BACK_PORCH    : integer := H_SYNC + 64;
5 constant H_PIXELS        : integer := 1040;
6
7 constant V_ACTIVE_VIDEO : integer := 600;
8 constant V_FRONT_PORCH  : integer := V_ACTIVE_VIDEO + 37;
9 constant V_SYNC         : integer := V_FRONT_PORCH + 6;
10 constant V_BACK_PORCH   : integer := V_SYNC + 23;
11 constant V_PIXELS       : integer := 666;
```

Procedimento incremental reutilizado

O mesmo padrão do timer reaparece no controlador VGA: incrementar um contador e informar estouro.

```
1 incrementa(horizontal, H_PIXELS, true, controle);  
2 incrementa(vertical, V_PIXELS, controle, controle);
```

A diferença é o significado dos contadores:

- horizontal: posição do pixel dentro da linha.
- vertical: posição da linha dentro do quadro.

Quando o contador horizontal estoura, o vertical avança uma linha.

Sincronismos e área ativa

O processo de varredura calcula os pulsos de sincronismo e a região visível.

```
1  if horizontal >= H_FRONT_PORCH and horizontal < H_SYNC then
2      VGA_HS <= '0';
3  end if;
4
5  if vertical >= V_FRONT_PORCH and vertical < V_SYNC then
6      VGA_VS <= '0';
7  end if;
8
9  if horizontal < H_ACTIVE_VIDEO and vertical < V_ACTIVE_VIDEO then
10     active <= true;
11 end if;
```

Esta parte é lógica de controle da tela. Ela não decide o que será desenhado; apenas informa onde o feixe lógico está.

Procedimento quadrado

O procedimento quadrado concentra a decisão geométrica de desenho.

```
1 procedure quadrado(  
2     constant enable : in boolean;  
3     constant h, v   : in integer;  
4     constant x, y   : in integer;  
5     constant w      : in integer;  
6     constant color  : in std_logic_vector(11 downto 0);  
7     variable draw   : inout std_logic_vector(11 downto 0)  
8 ) is  
9 begin  
10     if enable then  
11         if (v >= y - w/2 and v < y + w/2) and  
12             (h >= x - w/2 and h < x + w/2) then  
13             draw := color;  
14         end if;  
15     end if;  
16 end procedure;
```

O parâmetro draw é variable inout porque a cor pode ser modificada imediatamente dentro do processo de desenho.

Desenho com chamadas sucessivas

No processo de desenho, o fundo começa preto e cada chamada pode substituir a cor do pixel atual.

```
1 draw := (others => '0');
2
3 quadrado(active, horizontal, vertical, 350, 275, 60, x"F00", draw);
4 quadrado(active, horizontal, vertical, 400, 300, 60, x"0F0", draw);
5 quadrado(active, horizontal, vertical, 450, 325, 60, x"00F", draw);
6
7 VGA_R <= draw(11 downto 8);
8 VGA_G <= draw(7  downto 4);
9 VGA_B <= draw(3  downto 0);
```

A leitura fica direta: desenhe um quadrado vermelho, um verde e um azul, cada um com centro, tamanho e cor próprios.

Controle de varredura versus lógica de desenho

O exemplo VGA separa bem duas tarefas:

- **Varredura:** contar pixels e linhas, gerar `VGA_HS`, `VGA_VS` e `active`.
- **Desenho:** decidir a cor RGB para a coordenada atual.

Os procedimentos ajudam porque dão nome às operações repetidas: `incrementa` para contadores e `quadrado` para teste geométrico de região.

Observação didática: como `active` é um sinal atribuído no processo clockado, seu valor é atualizado após o ciclo de simulação atual. Em projetos de vídeo mais rigorosos, é comum alinhar sinais de controle e pixels com cuidado.

Atividades de laboratório

As atividades de laboratório estão no repositório da disciplina, nos arquivos Markdown da aula:

- a009g_lab_1.md: conversor Celsius para Fahrenheit usando função.
- a009h_lab_2.md: gerador PWM usando procedimento.

A sequência dos labs acompanha a aula: primeiro uma função com retorno único, depois um procedimento para uma rotina de controle reutilizável.

Como pensar os labs

- No conversor de temperatura, a função é adequada porque recebe Celsius e retorna Fahrenheit.
- No PWM, o procedimento é adequado porque descreve uma sequência de controle baseada em período, contador e duty cycle.
- Os dois labs devem começar com constantes reduzidas em simulação antes de usar valores adequados ao clock da placa.

Antes de levar para a placa, simule uma versão pequena. Reduzir constantes em testbench é uma estratégia legítima para observar comportamento sem esperar milhões de ciclos.

Resumo: função

Use função quando:

- a operação retorna um único valor;
- o código pode ser lido como uma expressão;
- não há necessidade de alterar sinais externos;
- a intenção é conversão, cálculo combinacional ou seleção de valor.

Exemplo da aula: `decodificando(valor)` retorna o padrão de sete segmentos para um dígito.

Resumo: procedimento

Use procedimento quando:

- há uma sequência de comandos reutilizável;
- a operação altera um sinal ou variável passado como parâmetro;
- há mais de uma informação de saída;
- a lógica tem caráter de ação: incrementar, testar estouro, desenhar, atualizar.

Exemplos da aula: `incrementa` no timer e no VGA; `quadrado` no processo de desenho.

Boas práticas

- Comece com subprogramas simples e próximos do problema real.
- Evite subprogramas genéricos demais no início; eles podem esconder a lógica que o aluno ainda precisa enxergar.
- Dê nomes que expressem comportamento: `incrementa`, `decodificando`, `quadrado`.
- Teste com frequências e limites reduzidos antes de usar os valores reais do hardware.
- Para síntese, prefira faixas explícitas quando usar inteiros.
- Mantenha a interface do módulo estável quando refatorar a implementação interna.

Subprogramas e síntese

Em software, uma função costuma ser imaginada como uma chamada executada no tempo.

Em VHDL sintetizável, subprogramas são principalmente uma forma de escrever a descrição do hardware:

- o sintetizador expande e interpreta a lógica descrita;
- chamadas repetidas podem representar hardware repetido ou lógica compartilhada conforme o contexto;
- o resultado final ainda depende de sinais, processos, clocks, resets e atribuições.

Portanto, subprogramas melhoram organização e reutilização, mas não substituem o raciocínio sobre o circuito.

Mensagem final

Nesta aula, funções e procedimentos apareceram como resposta a problemas concretos:

- a repetição do incremento no timer motivou um procedimento;
- a tabela do display motivou uma função;
- o VGA mostrou procedimentos separando varredura e desenho;
- os testbenches mostraram como validar comportamento sem depender imediatamente do hardware.

O objetivo é escrever VHDL mais claro sem perder de vista o circuito que está sendo descrito.